

Лабораторная №9. Работа с логическим анализатором

Цель работы

Изучить принципы работы логического анализатора, освоить управление GPIO через API `libgpiod 2.x` и получить практические навыки измерения цифровых сигналов и декодирования UART в Sigrok/PulseView.

Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с ALT Linux;
- USB-логический анализатор, поддерживаемый драйвером `fx2lafw`;
- соединительные провода;
- компьютер с Sigrok и PulseView;
- компилятор GCC, библиотека `libgpiod 2.x` и заголовочные файлы;
- доступ к UART TX и выводу PE16 платы.

На проверенной ревизии Lichee RV Dock используются следующие контакты:

Назначение	Вывод D1	Физический контакт
UART0 TX	PB8	8
тестовый GPIO	PE16	40
общий провод	GND	6

Перед подключением сверьте ревизию платы с её распиновкой. Входы анализатора должны допускать уровни 3,3 В.

Опасность повреждения

Собирайте и изменяйте схему только при выключенном питании. Соедините GND платы и анализатора, подключайте к GPIO только входной канал анализатора и не подавайте 5 В на выводы Lichee RV Dock.

Ключевые понятия

Определение

Логический анализатор — прибор, который дискретизирует логические уровни нескольких цифровых линий и записывает их изменения во времени.

Логический анализатор удобен для длительного многоканального захвата и декодирования цифровых протоколов. Осциллограф показывает фактическую форму и амплитуду сигнала, поэтому лучше подходит для анализа фронтов, выбросов, шумов и нарушений электрических уровней.

В ALT Linux доступны пакеты проекта Sigrok:

- sigrok-cli — утилита командной строки;
- pulseview — графический интерфейс.

Определение

Sigrok — проект программного обеспечения для работы с измерительными приборами и декодирования протоколов.

PulseView является графическим приложением проекта Sigrok для настройки захвата, отображения сигналов и работы с декодерами.

Определение

GPIO-линия — отдельный управляемый цифровой сигнал, идентифицируемый GPIO-чипом и смещением линии внутри него.

Дискретизация и алиасинг

Анализатор измеряет вход через равные интервалы времени. При частоте дискретизации f_s интервал между отсчётами равен $T_s = 1/f_s$. Например, при 1 МГц анализатор получает один отсчёт каждую микросекунду, а на бит UART 115200 бод приходится около $1000000/115200 = 8,68$ отсчёта.

Теорема Найквиста задаёт нижнюю границу в два отсчёта на период только для восстановления идеального периодического сигнала. Для распознавания коротких импульсов, измерения их длительности и декодирования протокола нужен запас. В этой работе используйте не менее 8–10 отсчётов на кратчайший значимый интервал. Для импульса 250 мкс достаточно 100 кГц, но 1 МГц даёт 250 отсчётов и удобнее для измерения.

Если частота дискретизации слишком мала, отдельные фронты могут попасть между отсчётами. Наблюдаемая частота и заполнение тогда отличаются от реальных — возникает алиасинг. Логический анализатор также не измеряет амплитуду, выбросы и форму фронта: для этого нужен осциллограф.

Запуск и триггер

Без триггера прибор начинает запись независимо от исследуемого события. Триггер по восходящему или нисходящему фронту выбранного канала делает начало захватов сопоставимым. Предыстория (pre-trigger) сохраняет часть данных до события и помогает увидеть состояние шины перед ним. Для периодического GPIO достаточно триггера по восходящему фронту; для UART удобно использовать нисходящий фронт стартового бита.

Глубина памяти ограничивает длительность наблюдения: при той же памяти увеличение частоты дискретизации уменьшает доступное время

захвата. Сначала выбирают кратчайший значимый интервал, затем частоту и длительность записи.

Порядок выполнения

Установка программного обеспечения

Все команды этого подраздела выполняются **на рабочем компьютере**, а не на Lichee. Установите программы для анализатора:

```
sudo apt-get install sigrok-cli sigrok-firmware-fx2lafw pulseview
```

Подключите анализатор к компьютеру и проверьте обнаружение:

```
lsusb  
sigrok-cli --scan
```

Проверка

Ожидаемый результат. В списке устройств присутствует анализатор с драйвером fx2lafw. Если устройство не найдено, проверьте USB-подключение, пакет прошивки и системный журнал.

Сохраните версии в отчёт:

```
sigrok-cli --version  
rpm -q sigrok-cli sigrok-firmware-fx2lafw pulseview
```

Подключение GPIO и настройка захвата

Выключите питание платы. Соедините GND анализатора с контактом 6 Lichee, а вход D0 — с контактом 40 (PE16). Не соединяйте PE16 с выходом или питанием анализатора. После проверки схемы включите питание и запустите PulseView **на рабочем компьютере**:

```
pulseview
```

Выберите анализатор fx2lafw, оставьте канал D0, задайте 1 МГц и захват не короче трёх секунд. Настройте триггер по восходящему фронту D0. Такой интервал содержит не менее 100 периодов режима 10/10 мс.

Эквивалентный захват из командной строки после запуска генератора:

```
sigrok-cli -d fx2lafw --config samplerate=1m \  
--channels D0 --triggers D0=r --time 3000 \  
--output-file gpio-10-10.sr
```

Имя канала и доступные частоты конкретного прибора предварительно проверьте командой `sigrok-cli --show -d fx2lafw`. Не перезаписывайте исходный `.sr` после получения результата.

Проверка

Проверка настройки. При 1 МГц на полуволну 10 мс приходится около 10000 отсчётов. Если фронты пропускаются, сначала проверьте общий GND и канал, затем частоту дискретизации и условие триггера.

Декодирование UART

UART-консоль Lichee RV Dock передаёт данные через PB8. При выключенном питании подключите:

- GND анализатора к контакту 6 платы;
- D1 анализатора к UART0 TX, контакту 8 (PB8).

В PulseView выполните следующие действия:

1. Оставьте подключённый канал.
2. Установите частоту дискретизации 2 МГц или выше.
3. Добавьте декодер UART.
4. Назначьте вход декодера каналу, подключённому к TX.
5. Установите параметры формата 115200 8N1.

```
Baud rate: 115200
Data bits: 8
Parity: none
Stop bits: 1
Bit order: lsb-first
Data format: ascii
Invert RX: no
Invert TX: no
Sample point (%): 50
```

С рабочего компьютера начните захват с триггером по нисходящему фронту D1. Затем в открытом UART-терминале Lichee выполните команду без перенаправления в устройство:

```
printf 'LAB9 UART 115200 8N1\n'
```

Терминал передаст введенную команду по RX, а оболочка выведет её результат по исследуемой линии TX. Альтернативный источник — загрузочные и системные сообщения при перезагрузке платы. Не записывайте в /dev/ttyS0 из второго процесса, пока этот порт обслуживает консоль.

Проверка

Ожидаемый результат. Декодер отображает LAB9 UART 115200 8N1. Если символы искажены, сначала проверьте общий GND, выбранный канал, скорость 115200 бод и отсутствие инверсии. Затем намерен-

но задайте в декодере 9600 бод, зафиксируйте искажение и верните 115200 бод.

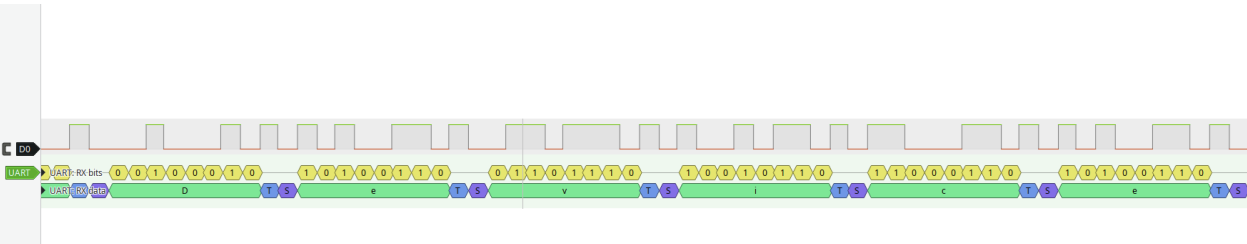


Рис. 1 - Декодирование загрузочного вывода UART 115200 8N1 в PulseView

Работа с libgpiod 2.x

Определение

libgpiod — библиотека пользовательского пространства для работы с символьным интерфейсом GPIO ядра Linux.

Следующие команды выполняются **на Lichee**. Пакет `libgpiod2` содержит библиотеку времени выполнения, а `libgpiod-devel` — заголовки и файлы для компоновки программ:

```
apt-get install gcc libgpiod2 libgpiod-devel
```

Проверьте установленную версию и доступные GPIO-чипы:

```
gpiodetect --version
gpiodetect
gpioinfo
```

Внимание

Синтаксис утилит и C API `libgpiod` 1.x и 2.x различается. Команды и код этой работы рассчитаны на `libgpiod` 2.x.

Основные объекты API 2.x:

- `struct gpiochip` представляет открытое символьное устройство `/dev/gpiochipN`;
- `struct gpio_line_settings` хранит направление и начальное значение линии;
- `struct gpio_line_config` связывает настройки со смещениями линий;
- `struct gpio_request_config` хранит параметры запроса, включая имя потребителя;
- `struct gpio_line_request` представляет полученный исключительный доступ к линиям.

Основные функции, используемые в каркасе программы:

```
#include <gpio.h>

struct gpio_chip *gpio_chip_open(const char *path);
void gpio_chip_close(struct gpio_chip *chip);

struct gpio_line_settings *gpio_line_settings_new(void);
int gpio_line_settings_set_direction(
    struct gpio_line_settings *settings,
    enum gpio_line_direction direction
);
int gpio_line_settings_set_output_value(
    struct gpio_line_settings *settings,
    enum gpio_line_value value
);
void gpio_line_settings_free(struct gpio_line_settings *settings);

struct gpio_line_config *gpio_line_config_new(void);
int gpio_line_config_add_line_settings(
    struct gpio_line_config *config,
    const unsigned int *offsets,
    size_t num_offsets,
    struct gpio_line_settings *settings
);
void gpio_line_config_free(struct gpio_line_config *config);

struct gpio_request_config *gpio_request_config_new(void);
void gpio_request_config_set_consumer(
    struct gpio_request_config *config,
    const char *consumer
);
void gpio_request_config_free(struct gpio_request_config *config);

struct gpio_line_request *gpio_chip_request_lines(
    struct gpio_chip *chip,
    struct gpio_request_config *req_config,
    struct gpio_line_config *line_config
);
int gpio_line_request_set_value(
    struct gpio_line_request *request,
    unsigned int offset,
    enum gpio_line_value value
);
void gpio_line_request_release(struct gpio_line_request *request);
```

В API 2.x для логических состояний применяются `GPIO_LINE_VALUE_INACTIVE` и `GPIO_LINE_VALUE_ACTIVE`, а не необозначенные целые значения.

Определение линии PE16

В используемой конфигурации PE16 соответствует смещению 144 в /dev/gpiochip0. Перед работой подтвердите это соответствие и убедитесь, что линия свободна:

```
gpioinfo -c 0 144
```

В строке линии не должно быть другого потребителя. Если утилиты libgpiod ещё не установлены, преподаватель может выполнить диагностическую проверку через debugfs:

```
grep -E 'gpiochip0|gpio-144' /sys/kernel/debug/gpio
```

Для ручной установки выхода утилитой gpiotest используется синтаксис 2.x:

```
gpiotest -c 0 144=1
```

Внимание

Номер /dev/gpiochip0 и смещение 144 необходимо проверять на фактически загруженной системе. Device Tree и версия ядра могут изменить представление GPIO; занятая линия не должна запрашиваться программой.

Генератор меандра

Сохраните программу как generator.c. Она проверяет два аргумента в микросекундах, устанавливает начальный низкий уровень и освобождает линию после Ctrl+C.

```
#include <gpiod.h>
#include <errno.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define GPIO_CHIP "/dev/gpiochip0"
#define GPIO_OFFSET 144U
#define MAX_DELAY_US 100000000ULL

static volatile sig_atomic_t keep_running = 1;

static void signal_handler(int signum)
{
    (void)signum;
    keep_running = 0;
}
```

```

}

static int parse_delay(const char *text, uint64_t *value)
{
    char *end;
    unsigned long long parsed;

    errno = 0;
    parsed = strtoull(text, &end, 10);
    if (errno || end == text || *end != '\0' || text[0] == '-' ||
        parsed == 0 || parsed > MAX_DELAY_US)
        return -1;

    *value = parsed;
    return 0;
}

static int sleep_us(uint64_t microseconds)
{
    struct timespec remaining = {
        .tv_sec = (time_t)(microseconds / 1000000ULL),
        .tv_nsec = (long)((microseconds % 1000000ULL) * 1000ULL),
    };

    while (keep_running && nanosleep(&remaining, &remaining) < 0) {
        if (errno != EINTR) {
            perror("nanosleep");
            return -1;
        }
    }
    return 0;
}

int main(int argc, char **argv)
{
    struct gpiod_request_config *request_config = NULL;
    struct gpiod_line_settings *settings = NULL;
    struct gpiod_line_config *line_config = NULL;
    struct gpiod_line_request *request = NULL;
    struct gpiod_chip *chip = NULL;
    const unsigned int offset = GPIO_OFFSET;
    uint64_t high_us, low_us;
    struct sigaction action = { .sa_handler = signal_handler };
    int status = EXIT_FAILURE;

    if (argc != 3 || parse_delay(argv[1], &high_us) < 0 ||
        parse_delay(argv[2], &low_us) < 0) {
        fprintf(stderr, "Использование: %s HIGH_US LOW_US\n"
            "Длительности: 1..%llu мкс\n",

```

```

        argv[0], MAX_DELAY_US);
    return EXIT_FAILURE;
}

sigemptyset(&action.sa_mask);
if (sigaction(SIGINT, &action, NULL) < 0 ||
    sigaction(SIGTERM, &action, NULL) < 0) {
    perror("sigaction");
    return EXIT_FAILURE;
}

chip = gpiod_chip_open(GPIO_CHIP);
if (!chip) {
    perror("gpiod_chip_open");
    goto out;
}

settings = gpiod_line_settings_new();
line_config = gpiod_line_config_new();
request_config = gpiod_request_config_new();
if (!settings || !line_config || !request_config) {
    fprintf(stderr, "Не удалось создать конфигурацию GPIO\n");
    goto out;
}

if (gpiod_line_settings_set_direction(
    settings, GPIOD_LINE_DIRECTION_OUTPUT) < 0 ||
    gpiod_line_settings_set_output_value(
    settings, GPIOD_LINE_VALUE_INACTIVE) < 0 ||
    gpiod_line_config_add_line_settings(
    line_config, &offset, 1, settings) < 0) {
    perror("Настройка GPIO");
    goto out;
}

gpiod_request_config_set_consumer(request_config, "wavegen");
request = gpiod_chip_request_lines(chip, request_config, line_config);
if (!request) {
    perror("gpiod_chip_request_lines");
    goto out;
}

printf("PE16: HIGH=%llu мкс, LOW=%llu мкс; Ctrl+C для остановки\n",
    (unsigned long long)high_us, (unsigned long long)low_us);
fflush(stdout);

while (keep_running) {
    if (gpiod_line_request_set_value(
        request, offset, GPIOD_LINE_VALUE_ACTIVE) < 0) {

```

```

        perror("Установка высокого уровня");
        goto out;
    }
    if (sleep_us(high_us) < 0 || !keep_running)
        break;
    if (gpio_line_request_set_value(
        request, offset, GPIO_LINE_VALUE_INACTIVE) < 0) {
        perror("Установка низкого уровня");
        goto out;
    }
    if (sleep_us(low_us) < 0)
        break;
}

status = EXIT_SUCCESS;

out:
    if (request) {
        if (gpio_line_request_set_value(
            request, offset, GPIO_LINE_VALUE_INACTIVE) < 0)
            perror("Сброс GPIO");
        gpio_line_request_release(request);
    }
    gpio_request_config_free(request_config);
    gpio_line_config_free(line_config);
    gpio_line_settings_free(settings);
    gpio_chip_close(chip);
    return status;
}

```

Скомпилируйте программу с библиотекой `libgpiod`:

```

gcc -std=c11 -D_POSIX_C_SOURCE=200809L -O2 \
    -Wall -Wextra -Wpedantic -Werror \
    generator.c -o generator -lgpiod

```

Проверка

Проверка сборки. GCC должен завершиться без ошибок и предупреждений, а команда `ldd ./generator` должна показывать зависимость от `libgpiod.so.2`.

Измерение генератора

Последовательно выполните на Lichee три режима. Для каждого получите отдельный захват не менее 100 полных периодов:

```

./generator 10000 10000
./generator 2000 8000
./generator 250 250

```

Одновременно должен работать только один экземпляр. Остановите его Ctrl+C, проверьте освобождение линии через `gpioinfo -c 0 144`, затем запускайте следующий режим.

До измерения рассчитайте ожидаемые значения по формулам

$$T = t_{HIGH} + t_{LOW}, \quad f = 1/T, \quad D = t_{HIGH}/T \cdot 100\%.$$

HIGH/LOW	Ожидаемый период	Ожидаемая частота	Заполнение
10/10 мс	20 мс	50 Гц	50%
2/8 мс	10 мс	100 Гц	20%
250/250 мкс	500 мкс	2 кГц	50%

В PulseView измерьте курсорами HIGH, LOW и период. По серии не менее 100 периодов определите минимум, максимум, среднее и стандартное отклонение периода. В этой работе джиттер «пик-пик» равен разности максимального и минимального периодов. Сохраните исходные .sr и таблицу результатов.

Опасность повреждения

Не соединяйте PE16, настроенный как выход, с другим активным выходом. Конфликт логических уровней может повредить оборудование.

Внимание

Пользовательский процесс Linux не обеспечивает жёсткий реальный масштаб времени. Планировщик, системные вызовы и функция ожидания вносят задержку и джиттер, поэтому измеренные интервалы могут отличаться от заданных, особенно при малых длительностях.

Проверка

Ожидаемый результат. Анализатор показывает периодический прямоугольный сигнал. После Ctrl+C программа завершается, а `gpioinfo` больше не показывает потребителя `wavegen` для выбранной линии.

На проверенном стенде Lichee RV Dock с Linux 6.16.0-spi-i2c-lab и анализатором Saleae Logic (fx2lafw) получены следующие физические измерения при 1 МГц:

Режим	Периодов	Средние Средний		Частота	Заполнение	σ_T	Джиттер
		HIGH/LOW	период				пик-пик
10/10 мс	145	10,241/10,240 мс	20,484 мс	48,819 Гц	49,995%	21 мкс	152 мкс
2/8 мс	284	2,244/8,249 мс	10,493 мс	95,305 Гц	21,386%	132 мкс	1300 мкс
250/250 мкс	3089	484,3/486,2 мкс	970,6 мкс	1030,319 Гц	49,902%	141 мкс	1705 мкс

Рост относительной ошибки и джиттера в коротком режиме показывает, почему пользовательский процесс Linux нельзя применять как точный генератор 2 кГц. Эти числа характеризуют конкретный прогон и не являются нормативом для другой нагрузки или версии ядра.

Отрицательные проверки

1. Запустите `./generator abc 10000`. Ожидаются подсказка и ненулевой код `echo $?`; линия не должна быть занята.
2. Запустите `./generator 10000 10000`, а из второго терминала повторите команду. Второй запрос должен завершиться ошибкой `Device or resource busy`. Остановите первый процесс и подтвердите освобождение линии.
3. Захватите режим 250/250 мкс с минимальной доступной частотой прибора, например 20 кГц. Сравните его с захватом 1 МГц: при 20 кГц временной шаг равен 50 мкс, поэтому границы импульсов и статистика квантуются грубее. Для явного алиасинга используйте UART 115200 бод при 20 кГц. После опыта верните рабочую частоту.
4. Для UART сравните декодирование при 9600 и 115200 бод, изменяя только скорость декодера.

Варианты SPI и I2C

После обязательных GPIO и UART выполните один вариант. Все соединения изменяйте при выключенном питании.

Вариант SPI. Используйте настроенный в лабораторной №7 loopback и подключите входы анализатора:

Канал	Сигнал	Вывод	Контакт
D0	CS	PD10	21
D1	SCK	PD11	22
D2	MOSI	PD12	19

Канал	Сигнал	Вывод	Контакт
D3	MISO	PD13	17

При частоте шины 1 МГц выберите частоту дискретизации не менее 10 МГц. Добавьте декодер SPI: CPOL=0, CPHA=0, MSB first, 8 бит, CS active-low. Запустите проверенную передачу `spidev_test` из лабораторной №7. Критерий: на каждую активную область CS приходится ожидаемое число тактов, а декодированные MOSI и MISO совпадают из-за loopback.

Вариант I2C. Используйте стенд SSD1306 из лабораторной №8: D0 подключите к SCL (PE12, контакт 38), D1 — к SDA (PG15, контакт 12). При стандартных 100 кГц установите частоту дискретизации не менее 2 МГц, добавьте декодер I2C и выполните на Lichee ограниченную проверку:

```
i2cdetect -y 2 0x3c 0x3c
```

Критерий: декодер показывает START, семибитный адрес 0x3C, направление записи, ACK и STOP. Адрес не следует записывать как 0x78: это байт адреса вместе с битом направления, а не семибитный адрес Linux.

На проверенном стенде захват при 2 МГц содержал 10 тактов SCL и транзакцию длительностью 263 мкс: START, 0x3C, WRITE, ACK, STOP. Постоянные HIGH без фронтов при успешном `i2cdetect` означали, что щупы анализатора были подключены не к фактическим линиям SCL/SDA.

Задание

1. Зафиксируйте версии Sigrok/PulseView и обнаружение анализатора.
2. Подтвердите PE16, /dev/gpiochip0, offset 144 и отсутствие потребителя.
3. Соберите генератор без предупреждений и проверьте ошибочный аргумент.
4. Снимите режимы 10/10 мс, 2/8 мс и 250/250 мкс с частотой 1 МГц.
5. Для каждого режима рассчитайте ожидаемые параметры и измерьте минимум, максимум, среднее и стандартное отклонение не менее 100 периодов.
6. Проверьте конфликт двух процессов, завершение по Ctrl+C и освобождение линии.
7. Повторите короткий режим при 20 кГц, объясните потерю временной точности и покажите алиасинг UART при этой частоте.
8. Декодируйте известную строку UART при 115200 8N1 и воспроизведите ошибку 9600 бод.
9. Выполните один вариант декодирования: SPI loopback или I2C с SSD1306.

10. Сохраните исходные захваты и продемонстрируйте результат преподавателю.

Контрольные вопросы

1. Для чего нужен логический анализатор при отладке цифровых сигналов?
2. Какие параметры необходимо настроить в PulseView для декодирования UART?
3. Что такое GPIO-линия в терминах libgpiod 2.x и как к ней обратиться из программы?
4. Чем управление GPIO через libgpiod отличается от прямой записи в регистры микроконтроллера?
5. Почему условие Найквиста недостаточно для надёжного декодирования цифрового протокола?
6. Что такое алиасинг и как отличить его от джиттера источника?
7. Для чего нужны триггер и предыстория захвата?
8. Почему второй процесс не может запросить уже занятую GPIO-линию?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- модель анализатора и вывод `sigrok-cli --scan`;
- настройки и результат декодирования UART;
- версию libgpiod и вывод, подтверждающий выбранные GPIO-чип и смещение;
- исходный код и команду сборки генератора;
- параметры запуска и сохранённый захват PulseView;
- таблицу заданных и измеренных длительностей, периода, частоты и заполнения;
- статистику минимум по 100 периодам и определение использованной метрики джиттера;
- результаты четырёх отрицательных проверок;
- декодированный захват выбранного варианта SPI или I2C;
- описание завершения программы и освобождения линии;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.